Vector – ulepszona tablica

vector jest strukturą, która na pierwszy rzut oka wygląda jak zwykła tablica. Oprócz funkcjonalności zwykłej tablicy vectory posiadają wiele przydatnych funkcji i ulepszeń.

Przypomnijmy więc, jakie zastosowanie w programowaniu ma tablica. Załóżmy, że musisz przechować milion liczb całkowitych. Przygotuj więc milion pudełek, gdzie w każdym pudełku może znajdować się jedna taka liczba i ponumeruj je od 0 do milion - 1. Pudełka te znajdują się w pokoju o ustalonej nazwie — tą nazwą jest nazwa tablicy. Teraz jedynie co musisz wiedzieć, to jak jej używać. Wystarczy, że podasz nazwę pokoju (nazwę tablicy), a w nawiasie kwadratowym podasz numer pudełka (numer komórki tablicy).

W praktyce wektory działają na tej samej zasadzie, tylko że nie musisz z góry określać liczby pudełek, możesz je dokładać i usuwać w dowolnym momencie. Co więcej, od razu możesz przypisać do każdego pudełka ustaloną liczbę. Nie musisz przejmować się alokacją pamięci, ponieważ dodając kolejne elementy, pamięć będzie rozszerzana automatycznie. Aby korzystać z wektorów, należy w nagłówku programu dołączyć bibliotekę

```
#include <vector>
```

Tworzenie vectora

Możesz utworzyć pusty wektor, bez przypisanej liczby elementów (sam pokój, bez pudełek). W miarę potrzeb możesz dokładać elementy na koniec wektora (dokładać kolejne pudełka do pokoju).

```
vector <typ elementów wektora> nazwa_wektora;
```

Np.:

```
vector <int > tab;
```

Jeśli jednak potrzebujesz na początku określić liczbę elementów wektora, możesz to zrobić w następujący sposób:

```
vector <typ elementów wektora> nazwa_wektora(liczba_elementów);
```

Np.:

```
vector <int > tab(1000000);
```

W sytuacji, gdy określasz liczbę elementów w wektorze, możesz od razu przypisać do każdego z nich ustaloną wartość:

```
vector <typ elementów wektora> nazwa_wektora(liczba_elementów, ustalona_wartość);
```

Np.:

```
vector <int > tab(1000000, 1);
```

Podczas inicjacji wektora możesz zdefiniować zwoje wartości, które chcesz, aby były przypisane do kolejnych pozycji:

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {
7      {setlocale(LC_CTYPE, "Polish");
8      vector <int> liczby = {1, 2, 3, 4, 5};
9
10     //wypisanie elementów wektora
11     for(int i=0; i<liczby.size(); i++)
12         cout<<liczby[i]<<' ';
13
14     return 0;
15 }
```

Dodawanie kolejnych elementów do wektora

Aby dodać nowy element na koniec wektora, należy użyć metody `push_back(element);`. Aby użyć tej metody, musisz podać nazwę wektora, a następnie po kropce wpisać `push_back(element)` i w nawiasie podać wartość elementu np.:

```
vector <int > tab;
tab.push_back(3);
int a = 4;
tab.push_back(a);
```

W pierwszym pudełku o numerze 0 będzie przechowywana liczba 3, natomiast w pudełku o numerze 1 będzie liczba 4.

Odwoływanie się do elementów wektora

Aby sprawdzić, jaka wartość znajduje się w pudełku o zadanym numerze (indeksie), należy użyć operatora `[ ]`— nawiasu kwadratowego, a wewnątrz niego należy wpisać numer komórki (musi istnieć komórka o zadanym indeksie).

```
vector <int > tab;
tab.push_back(3);
int a = 4;
tab.push_back(a);
cout<>tab[0]; //przypisanie nowej wartości do komórki o numerze 0
```

Przykład 1

Napisz program, który będzie pobierał liczby całkowite do momentu napotkania wartości zero, a następnie wypisze te liczby (bez zera) w odwróconej kolejności

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {
7      vector <int> tab;
8      int a;
9
10     cin>>a;
11     while(a != 0)
12     {
13         //dodanie elementu do wektora
14         tab.push_back(a);       //np. 1 2 3 0
15         cin>>a;
16     }
17
18     //wypisanie liczb w odwróconej kolejności
19     for(int i = tab.size()-1; i >= 0; i--)
20         cout<<tab[i]<<' '; //3 2 1
21
22                                 //sposób II wpisania liczb w odwróconej kolejności
23     while(!tab.empty())         //dopóki są jakieś elementy w wektorze
24     {
25         cout<<tab.back()<<' ';  //wypisanie elementu stojącego na końcu wektora
26         tab.pop_back();         //usunięcie ostatniego elementu w wektorze
27     }
28     return 0;
29 }
```

Uwaga! Metoda size() zwraca liczbę elementów w wektorze.

Metody:

Przydatne metody

Back

back — zwraca ostatni element wektora

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7      char znak;
8      vector <char> znaki;
9      cout<<"Podaj 5 znaków: "<<endl;
10     for(int i=0; i<5; i++) //np: abcde
11     {
12         cout<<"znak "<<i<<":";
13         cin>>znak;
14         znaki.push_back(znak);
15     }
16     //wypisanie ostatniego znaku
17     cout<<"ostatni znak to:"<<znaki.back();
18     return 0;
19 }
```

begin

begin — zwraca iterator wskazujący na pierwszy element

clear

clear — czyści zawartość wektora

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7      char znak;
8      vector <char> znaki;
9      cout<<"Podaj 5 znaków: ";
10     for(int i=0; i<5; i++) //np: abcde
11     {
12         cin>>znak;
13         znaki.push_back(znak);
14     }
15     //wypisanie liczby elementów w wektorze
16     cout<<"W wektorze znajduje się "<<znaki.size()<<" elementów.\n"; //W wektorze znajduje się 5 elementów.
17     //czyszczenie zawartości wektora
18     znaki.clear();
19     cout<<"W wektorze znajduje się "<<znaki.size()<<" elementów."; //W wektorze znajduje się 0 elementów.
20     return 0;
21 }
```

emplace

emplace — dodaje nowy element w dowolne miejsce w wektorze

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7      vector <int> liczby;
8      vector <int>::iterator miejsce;
9      int ktory=0;
10
11     //wstaw kolejno liczby: 1, 2, 3, 4 do wektora o nazwie liczby
12     liczby.push_back(1);
13     liczby.push_back(2);
14     liczby.push_back(3);
15     liczby.push_back(4);
16
17     //wyswietlenie
18     cout<<"wektor:"<<endl;
19     for(int i=0; i<liczby.size(); i++) cout<<liczby[i]<<' ';
20
21     cout<<endl<<"wielkość wektora to "<<liczby.size()<<endl;
22     cout<<"podaj miejsce (index) do wstawienia wartosci 99 - od 0 do "<<liczby.size()-1<<endl;
23     cin>>ktory;
24     miejsce=liczby.begin()+ktory;
25     liczby.emplace(miejsce,99);
26
27     //wyswietlenie
28     cout<<"wektor:"<<endl;
29     for(int i=0; i<liczby.size(); i++) cout<<liczby[i]<<' ';
30
31     return 0;
32 }
```

empty

empty — sprawdza, czy w wektorze są jakieś elementy

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7      vector <double> liczby;
8
9      if(liczby.empty() == true) //lub if(liczby.empty())
10         cout<<"W wektorze nie ma liczb."<<endl; //wykona się ta część instrukcji warunkowej
11     else
12         cout<<"W wektorze jest co najmniej jedna liczba."<<endl;
13
14     liczby.push_back(99);
15
16     if(liczby.empty() == true)
17         cout<<"W wektorze nie ma liczb.";
18     else
19         cout<<"W wektorze jest co najmniej jedna liczba."; //wykona się ta część instrukcji warunkowej
20
21     return 0;
22 }
```

end

end — zwraca iterator wskazujący na ostatni element

erase

erase — usunięcie wybranych elementów wektora

Metoda `erase()` usuwa element z dowolnego miejsca z wektorze, przesuwając o jeden w lewo elementy stojące za nim, zmniejszając odpowiednio wielkość wektora. Robi to w czasie liniowym w stosunku do elementów, które musi przesunąć. Do usunięcia danych z wektora wykorzystuje `iterator`, który należy ustawić na pozycję elementu do usunięcia. Jeśli jest potrzeba usunięcia pierwszej lub ostatniej wartości, możemy skorzystać z metod `begin()` i `end()`, które zwracają odpowiednio `iterator` wskazujący na pierwszy i ostatni element wektora.

Przykład 1

Zadanie. Wstaw do wektora liczby całkowite: 1 2 3 4 5, następnie usuń liczbę 2 i 4. Wypisz zawartość wektora przed i po usunięciu liczb.

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7      vector <int> liczby;
8      vector <int>::iterator it;
9
10     //wstaw kolejno liczby: 1, 2, 3, 4 i 5 do wektora o nazwie liczby
11     liczby.push_back(1);
12     liczby.push_back(2);
13     liczby.push_back(3);
14     liczby.push_back(4);
15     liczby.push_back(5);
16
17     for(int i=0; i<liczby.size(); i++)
18         cout<<liczby[i]<<' ';
19
20     cout<<endl;
21
22     //ustaw iterator na drugi element wektora
23     it = liczby.begin() + 1;
24     //usunięcie liczby 2
25     liczby.erase (it);
26
27     //ustaw iterator na trzeci element wektora
28     it = liczby.begin() + 2;
29     //usunięcie liczby 4
30     liczby.erase (it);
31
32     for(int i=0; i<liczby.size(); i++)
33         cout<<liczby[i]<<' ';
34
35     return 0;
36 }
```

Przykład 2

Zadanie. Wstaw do wektora liczby całkowite: 1 2 3 4 5, następnie usuń pierwszą i ostatnią wartość. Wypisz zawartość wektora przed i po usunięciu liczb.

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7
8      vector <int> liczby;
9
10     //wstaw kolejno liczby: 1, 2, 3, 4 i 5 do wektora o nazwie liczby
11     liczby.push_back(1);
12     liczby.push_back(2);
13     liczby.push_back(3);
14     liczby.push_back(4);
15     liczby.push_back(5);
16     cout<<"rozmiar wektora ="<<liczby.size()<<endl;
17     for(int i=0; i<liczby.size(); i++)
18         cout<<liczby[i]<<' ';
19
20     cout<<endl;
21
22     //usunięcie pierwszej liczby
23     liczby.erase (liczby.begin());
24     cout<<"rozmiar wektora po usunieciu pierwszego elementu="<<liczby.size()<<endl;
25         for(int i=0; i<liczby.size(); i++)
26         cout<<liczby[i]<<' ';
27         cout<<endl;
28     //usunięcie ostatniej liczby
29     liczby.erase(liczby.end()-1);
30
31
32     cout<<"rozmiar wektora po usunieciu ostatniego elementu="<<liczby.size()<<endl;
33
34     for(int i=0; i<liczby.size(); i++)
35         cout<<liczby[i]<<' ';
36
37     return 0;
38 }
```

front

front — zwraca pierwszy element wektora

Metoda **front()** zwraca pierwszy element w wektorze. Robi to w czasie stałym *O*(1). Jest to alternatywa dla odwołania się do pierwszego elementu za pomocą nawiasu kwadratowego z indeksem 0. Przed użyciem zadbaj, aby w wektorze wystąpiła co najmniej jedna wartość.

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7
8      vector <int> liczby;
9      liczby.push_back(3);
10     liczby.push_back(4);
11     liczby.push_back(1);
12
13     cout<<"Pierwsza liczba w wektorze to: "<<liczby[0]
14     <<". Pierwszą liczbę możemy także odczytać przy pomocy metody front: "<<liczby.front();// 3 3
15
16     return 0;
17 }
```

insert

insert — wstawia nowe elementy do wektora

Metoda **insert()** wstawia elementy z jednego wektora lub tablicy w dowolne miejsce do drugiego wektora, przesuwając elementy stojące za nim w prawo i rozszerzając odpowiednio wielkość wektora. Wstawienie to jest wykonane w czasie liniowym w stosunku do elementów, które musi przesunąć. Funkcja ta może także wstawiać pojedynczą wartość na podobnej zasadzie jak funkcja **emplace(element)**. W przypadku, gdy wstawiana jest zawartość całego wektora lub tablicy, należy wywołać tę metodę z trzema argumentami:

```
nazwa_wektora.insert([wskaźnik na element, przed który wstawiać będziemy zawartość obiektu], [wskaźnik na początkowy element w wektorze lub tablicy], [wskaźnik na końcowy element w wektorze lub tablicy];
```

W przypadku wstawienia pojedynczego elementu należy podać dwa argumenty: wskaźnik na element, przed który wstawiać będziemy element oraz wartość tego elementu.

Przykład 1

Zadanie. Wstaw do wektora pierwszego liczby całkowite: 1 2 3 4 5, następnie wstaw liczby 6 7 8 9 do drugiego wektora. Zawartość drugiego wektora wstaw między wartością 1 i 2. Następnie wstaw liczbę 100 na początek i koniec wektora pierwszego. Wypisz jego zawartość po wykonaniu wszystkich czynności.

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  int main()
6  {   setlocale(LC_CTYPE, "Polish");
7
8      //stworzenie wektorów i nadanie im wartości początkowych
9      vector <int> tab1 = {1, 2, 3, 4, 5}, tab2 = {6, 7, 8, 9};
10
11     //wstawienie zawartości wektora drugiego do wektora 1 między liczby 1 i 2
12     tab1.insert(tab1.begin()+1, tab2.begin(), tab2.end());
13
14     //wstawienie na początek liczbę 100
15     tab1.insert(tab1.begin(), 100);
16     //wstwienie na koniec liczbę 100
17     tab1.insert(tab1.end(), 100);
18
19     //wypisanie zawartości wektora
20     for(int i=0; i<tab1.size(); i++)
21         cout<<tab1[i]<<' ';
22
23     return 0;
24 }
```

```
C:\Users\Main\Desktop\Untitled3.exe
100 1 6 7 8 9 2 3 4 5 100
--------------------------------
Process exited after 0.02816 seconds with return value 0
Press any key to continue . . .
```

Przykład 2

Zadanie. Wstaw do wektora pierwszego liczby całkowite: 1 2 3 4 5, następnie wstaw liczby 6 7 8 9 do zwykłej tablicy. Wypisz zawartość wektora po wykonaniu wszystkich czynności.

```
1   #include<iostream>
2   #include<vector>
3   using namespace std;
4
5   int main()
6   {   setlocale(LC_CTYPE, "Polish");
7
8       //stworzenie wektora i nadanie im wartości początkowych
9       vector <int> tab1 = {1, 2, 3, 4, 5};
10
11      //inicjacja tablicy liczb całkowitych
12      int tab2[4] = {6, 7, 8, 9};
13
14      //wstawienie zawartości tablicy do wektora między liczby 1 i 2
15      tab1.insert(tab1.begin()+1, tab2, tab2+4);
16
17      //wypisanie zawartości wektora
18      for(int i=0; i<tab1.size(); i++)
19          cout<<tab1[i]<<' ';
20
21      return 0;
22  }
```

iterator

iterator — poruszanie się po elementach wektora

pop_back

pop_back — usuwa ostatni element z wektora

Metoda, która usuwa jeden element z kontenera vector, który znajduje się na jego końcu. Niniejszej metody nie można wywoływać, gdy kontener jest pusty.

```
1   #include<iostream>
2   #include<vector>
3   using namespace std;
4
5   int main()
6   {   setlocale(LC_CTYPE, "Polish");
7
8       vector < int > dane;
9       dane.push_back( 5 );
10      dane.push_back( 4 );
11      dane.push_back( 3 );
12      dane.push_back( 2 );
13      dane.pop_back();
14
15      //wypisanie zawartości wektora
16      for(int i=0; i<dane.size(); i++)
17          cout<<dane[i]<<' ';
18
19      return 0;
20  }
```

push_back

push_back — wstawia nowy element na koniec wektora

Metoda **push_back(element)** dostawia kolejny element na koniec wektora, rozszerzając automatycznie jego rozmiar. Metodę tę wywołujemy, podając nazwę wektora a następnie po kropce podajemy nazwę metody **push_back(element)**.

reverse

reverse — odwraca kolejność elementów w wektorze

```
1  #include<iostream>
2  #include<vector>
3  #include<algorithm> // !!!
4  using namespace std;
5
6  int main()
7  {   setlocale(LC_CTYPE, "Polish");
8
9      vector < int > dane;
10     dane.push_back( 5 );
11     dane.push_back( 4 );
12     dane.push_back( 3 );
13     dane.push_back( 2 );
14     //wypisanie zawartości wektora
15     for(int i=0; i<dane.size(); i++)
16         cout<<dane[i]<<' ';
17     cout<<endl;
18     reverse(dane.begin(),dane.end());
19     //wypisanie zawartości wektora
20     for(int i=0; i<dane.size(); i++)
21         cout<<dane[i]<<' ';
22     return 0;
23 }
```

size

size — zwraca liczbę elementów w wektorze

swap

swap — zamienia ze sobą wartości dwóch wektorów

Dane z kontenera przekazanego poprzez argument trafiają do kontenera, który wywołał niniejszą metodę, natomiast dane z kontenera wywołującego niniejszą metodę trafiają do kontenera przekazanego poprzez argument.

```
1  #include<iostream>
2  #include<vector>
3  #include<algorithm> // !!!
4  using namespace std;
5
6  int main()
7  {   setlocale(LC_CTYPE, "Polish");
8
9      vector < int > dane;
10     dane.push_back( 1 );
11     dane.push_back( 2 );
12     dane.push_back( 3 );
13     dane.push_back( 4 );
14
15     vector < int > dane2;
16     dane2.push_back( 11 );
17     dane2.push_back( 12 );
18     dane2.push_back( 13 );
19     dane2.push_back( 14 );
20     //wypisanie zawartości wektora
21     cout<<"dane dane2"<<endl;
22     for(int i=0; i<dane.size(); i++)
23         cout<<dane[i]<<"    "<<dane2[i]<<endl;
24     //zamiana
25     dane.swap(dane2);
26     cout<<"dane dane2"<<endl;
27     for(int i=0; i<dane.size(); i++)
28         cout<<dane[i]<<"    "<<dane2[i]<<endl;
29     return 0;
30 }
```